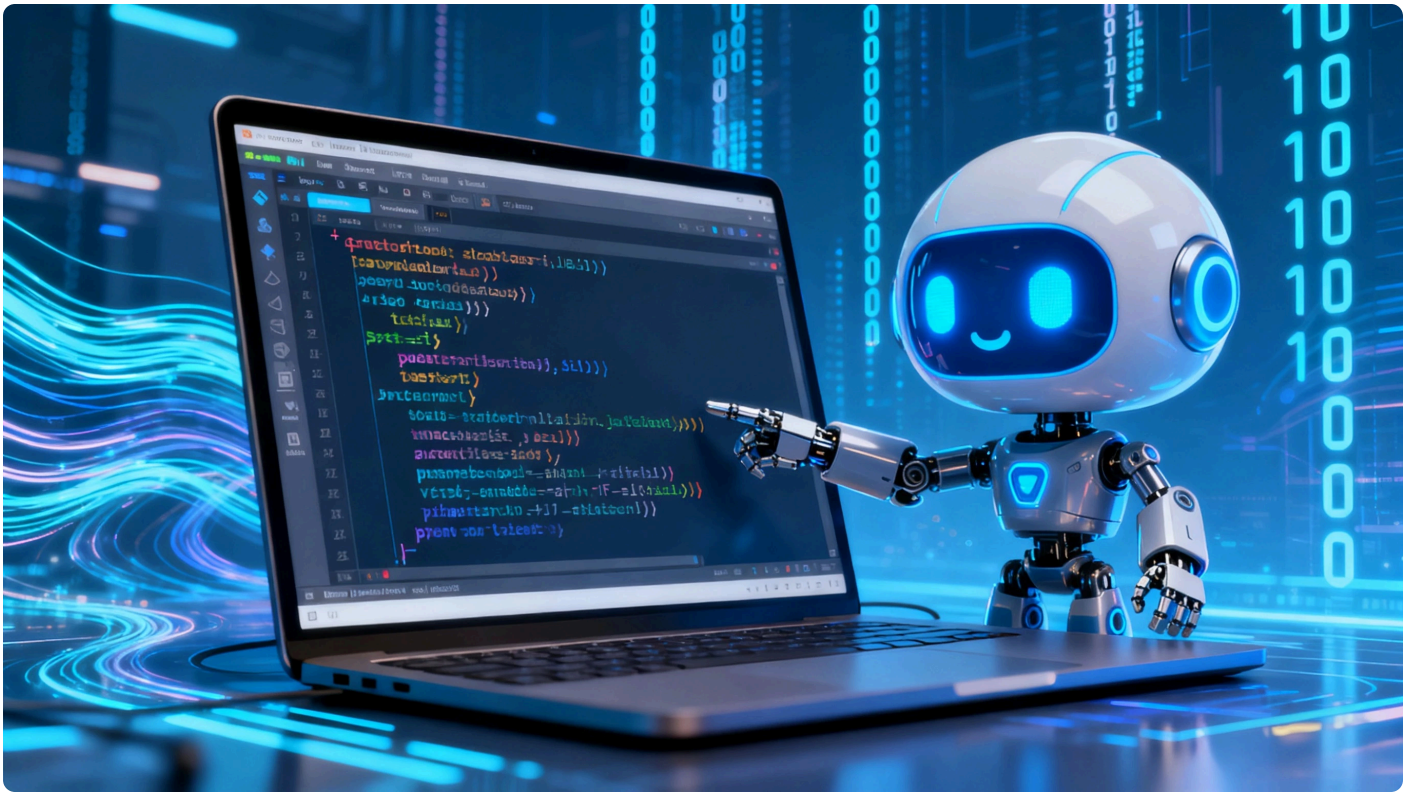
08 | 自定义指令：精通Slash Commands，打造你的私人命令集

Tony Bai · AI 原生开发工作流实战



你好，我是 Tony Bai。

在前面的几讲中，我们已经学会了如何通过 `@` 指令为 AI 提供精准的上下文，以及如何通过 `CLAUDE.md` 和 `constitution.md` 为它注入持久的“记忆”和“原则”。现在，我们的 AI 伙伴已经足够“博学”，能够“懂你项目”了。

但是，一个新的效率瓶颈又出现了。你可能会发现，自己每天都在输入一些类似但冗长的指令：

> “请帮我审查一下 `@internal/api/` 这个包里的 Go 代码，重点关注错误处理和是否符合我们的 `constitution.md`。”
>
> “请为我刚才写的 `@utils/parser.go` 这个文件生成一份单元测试，确保覆盖所有分支，并使用表格驱动的风格。”

> “请分析 `!git diff --staged` 的输出，并为我生成一条符合 Conventional Commits 规范的提交信息。”

这些指令虽然有效，但每次都手动输入，不仅繁琐，而且容易出错。它们就像 Shell 脚本出现之前，我们每次都要手动敲打一长串的 `find | xargs | grep` 组合一样。**有没有一种方法，可以将这些高频、复杂的、蕴含了我们最佳实践的工作流，封装成一个简单的、可复用的指令呢？**

答案是肯定的。这正是我们今天要精通的核心工具——**Slash Commands（斜杠指令）**。正如 Anthropic 在其官方最佳实践指南中提到的，对于重复性的工作流——无论是调试、日志分析还是代码审查——将提示模板存储为 Slash Commands 是提升效率的关键。

今天这一讲，我们将系统地学习 Claude Code 的指令体系。先熟悉那些能极大提升日常效率的**内置指令**，然后深入学习如何创造属于我们自己和团队的**自定义指令**。学完之后，你将完成一次关键的角色转变：从一个被动向 AI 提问的“使用者”，进化为一个主动为 AI 定义能力的“**指令设计师**”。

内置指令精讲：日常提效的瑞士军刀

Claude Code 内置了大约二十多个 Slash Commands，它们就像一把功能丰富的瑞士军刀，为我们的日常 AI 协作提供了极大的便利。我们不必逐一记忆，只需掌握其中最高频、最有价值的几个，就能让生产力倍增。

我将这些核心指令分为四大类：**会话与上下文管理**、**环境与配置**、**项目与协作**，以及**元信息与帮助**，并为你逐一深度剖析。

会话与上下文管理类：掌控你的“AI 记忆”

这类指令是最高频的，它们的核心是管理 AI 的“短期工作记忆”，确保 AI 始终聚焦于正确的任务。

指令	作用	解读	实践
/clear	**重置AI的上下文窗口，并清空屏幕。** 这是每个Claude Code用户都必须掌握的**第一个、也是最重要的指令**。	在一个长会话中，AI的上下文窗口会填满大量的对话历史、文件内容（通过@加载）和命令输出。这不仅会消耗更多的Token（增加成本），还可能因为无关信息的干扰而降低AI的响应性能和准确性，导致它“犯糊涂”。/clear就像是按下了AI记忆的“重置按钮”，让你可以**在一个干净的上下文中开始一个全新的、不相关的任务**	在任务之间频繁地使用/clear。当你完成了一个Bug修复，准备开始一个新功能开发时；或者当你从代码审查切换到文档撰写时，都应该果断使用/clear
/compact [instructions]	智能地“压缩”当前对话历史，保留核心信息，然后在一个新的、更轻量的对话分支中继续	如果说/clear是“格式化硬盘”，那么/compact就是“磁盘清理”。它适合在**同一个复杂任务的不同阶段之间**使用	比如，你和AI花了很多轮次讨论API的设计，现在准备进入实现阶段，就可以运行/compact 重点保留最终确定的API接口定义，这样既减轻了上下文负担，又保留了最重要的结论
/rewind (Esc Esc)	打开“时光机”，让你回到之前的某个对话节点	这是我们在第10讲才会深入讲解的Checkpointing功能的前端入口	当AI的一次代码重构不如人意时，/rewind可以让你精确地将**代码和/或对话历史**一键恢复到修改前的状态

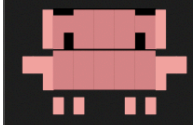
极客时间

环境与配置类：你的 AI“控制面板”

这类指令让你能够实时地查看和修改 Claude Code 的运行配置，将其调校到最称手的状态。

指令	作用	场景
/config	打开一个可视化的、交互式的设置界面	这是进行所有持久化配置的总入口。想换个主题？想修改默认的AI模型？想配置Vim模式？想管理Hooks？直接输入/config即可
/permissions	打开一个交互式菜单（如下图），用于管理**AI的工具“白名单”**	Claude Code的设计哲学是安全优先。默认情况下，任何可能修改你系统的操作（如写文件、执行git commit）都需要你手动批准。对于一些你完全信任的高频操作，/permissions允许你将其加入“白名单”，从而实现“一次授权，永久自动执行”。这是在安全和效率之间找到平衡的关键。我们将在第09讲深入探讨它
/model [model]	快速查看或切换当前会话使用的AI模型	动态的成本与性能控制器。处理简单任务（如代码格式化）时，用/model haiku切换到“haiku（对应glm-4.5-air）”；进行复杂架构设计时，用/model default切换到默认的高级模型（glm-4.6）

极客时间



Claude Code v2.0.15
glm-4.6 · API Usage Billing
/root/go/src/github.com/bigwhite/issue2md

> /config

Settings: Status **Config** Usage (tab to cycle)

Configure Claude Code preferences

❯ Auto-compact	true
Show tips	true
Rewind code (checkpoints)	true
Verbose output	false
Theme	Dark mode
Notifications	Auto
Output style	default
Editor mode	normal
Model	Default (recommended)
Auto-connect to IDE (external terminal)	false

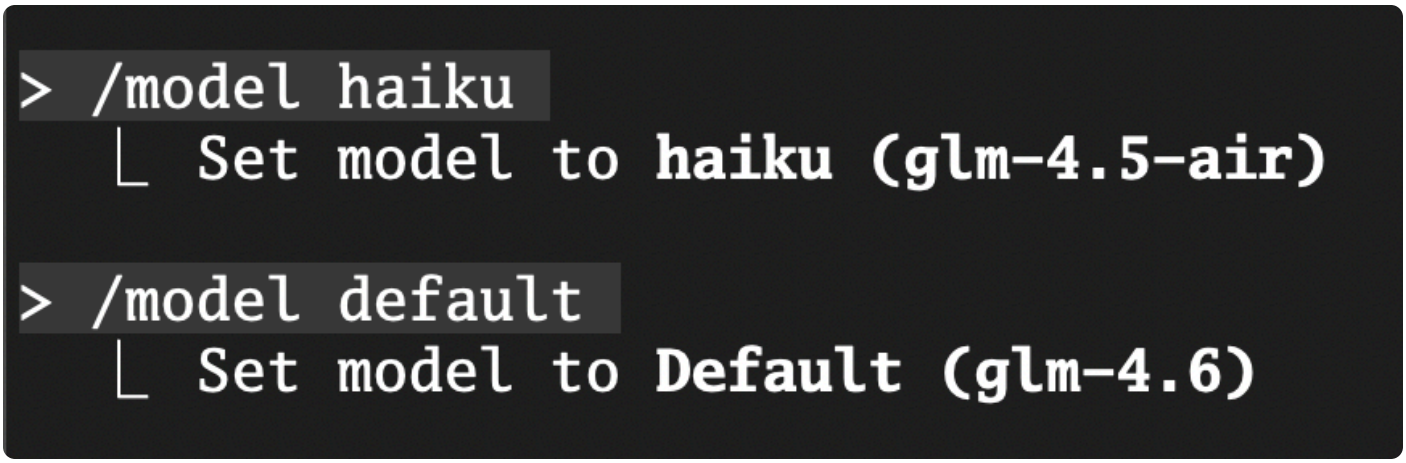
Enter/Space to change · Esc to exit

> /permissions

Permissions: **Allow** Ask Deny Workspace (tab to cycle)
Claude Code won't ask before using allowed tools.

❯ 1. Add a new rule…
  2. Bash(go list:*)

Enter to confirm · Esc to cancel



项目与协作类：将 AI 融入工程实践

这类指令与具体的软件工程任务紧密相关。

指令	作用	场景
/init	在06讲提到过，该指令在当前目录**自动生成一个CLAUDE.md文件的模板**	当你开始一个新项目，或想为一个现有项目引入AI协作规范时，这是最快的启动方式。它为你提供了一个结构良好的起点，你只需填充团队的具体规范即可
/memory	在06讲也详细说明过，该指令负责对选定的CLAUDE.md进行记忆查看以及系统性的、复杂的编辑和重构	这也是一个至关重要的**调试工具**。当你怀疑AI Agent没有遵循项目规范时，输入/memory，可以立刻看到AI“脑中”的规则到底是什么，从而快速定位问题
/review	这是一个内置的、专门用于请求代码（Pull Request, PR）审查的指令。它通常会查找项目的PR列表（通过github gh命令行工具），并触发一个更专注、更细致的审查流程	虽然你也可以用自然语言说“请帮我审查代码”，但使用/review的意图更明确，可能会让AI调用一个专门的审查Sub-agent（我们将在进阶篇学习），从而得到更专业的反馈
/pr_comments	获取并显示当前关联的Pull Request中的所有评论（通过gh pr view --json number,headRepository命令获取PR评论）	当你在一个PR分支上工作，但不想切换到浏览器去查看同事的审查意见时，可以直接运行/pr_comments。AI会将所有评论拉取到你的终端，并将其作为上下文。紧接着，你就可以说：“根据上面的第二条评论，帮我修改@main.go文件。”

极客时间

元信息与帮助类：你的实时“说明书”

这类指令帮助你随时了解工具的状态和获取帮助。

指令	作用	场景
/help	Claude Code帮助的入口，显示所有可用的Slash Commands列表，包括内置的和所有自定义的	任何时候，当你忘记了一个指令的名字，或者想探索一下当前项目或你的个人配置中又增加了哪些新能力时，输入/help或直接输入/再按Tab，总能给你答案
/status	快速显示一个包含Claude Code版本、当前模型、账户信息和网络连接状态的“状态面板”	当你感觉AI响应变慢或出现异常时，/status可以帮你快速诊断问题。例如，你可以看到当前连接的API端点是否正确
/doctor	运行一系列的自检程序，检查Claude Code的安装是否健康，依赖是否完整，配置是否正确	当你遇到一些无法解释的奇怪问题时，运行/doctor就像是给Claude Code做了一次“全身CT”。它能帮你发现一些潜在的环境问题
/cost 和/usage	/cost显示**当前会话**的Token消耗和预估费用。/usage（订阅用户可用）显示你整个**账户周期**内的套餐用量和速率限制状态	/cost是你的“单次任务成本计”，适合用来评估不同任务的开销。/usage则是你的“账户总览仪表盘”，适合用来管理整体预算和API调用频率
/feedback (or /bug)	打包当前会话的上下文（已做脱敏处理），并引导你到一个可以向Anthropic官方报告Bug的流程	当你确信遇到了一个Claude Code本身的Bug，而不是模型行为问题时，使用它来帮助官方改进产品

极客时间



```
> /status

Settings: Status  Config  Usage  (tab to cycle)

Version: 2.0.15
Session ID: e250eaf4-aa81-4485-b0b5-831e903e4d8c
cwd: /root/go/src/github.com/bigwhite/issue2md
Auth token: ANTHROPIC_AUTH_TOKEN
Anthropic base URL: https://open.bigmodel.cn/api/anthropic

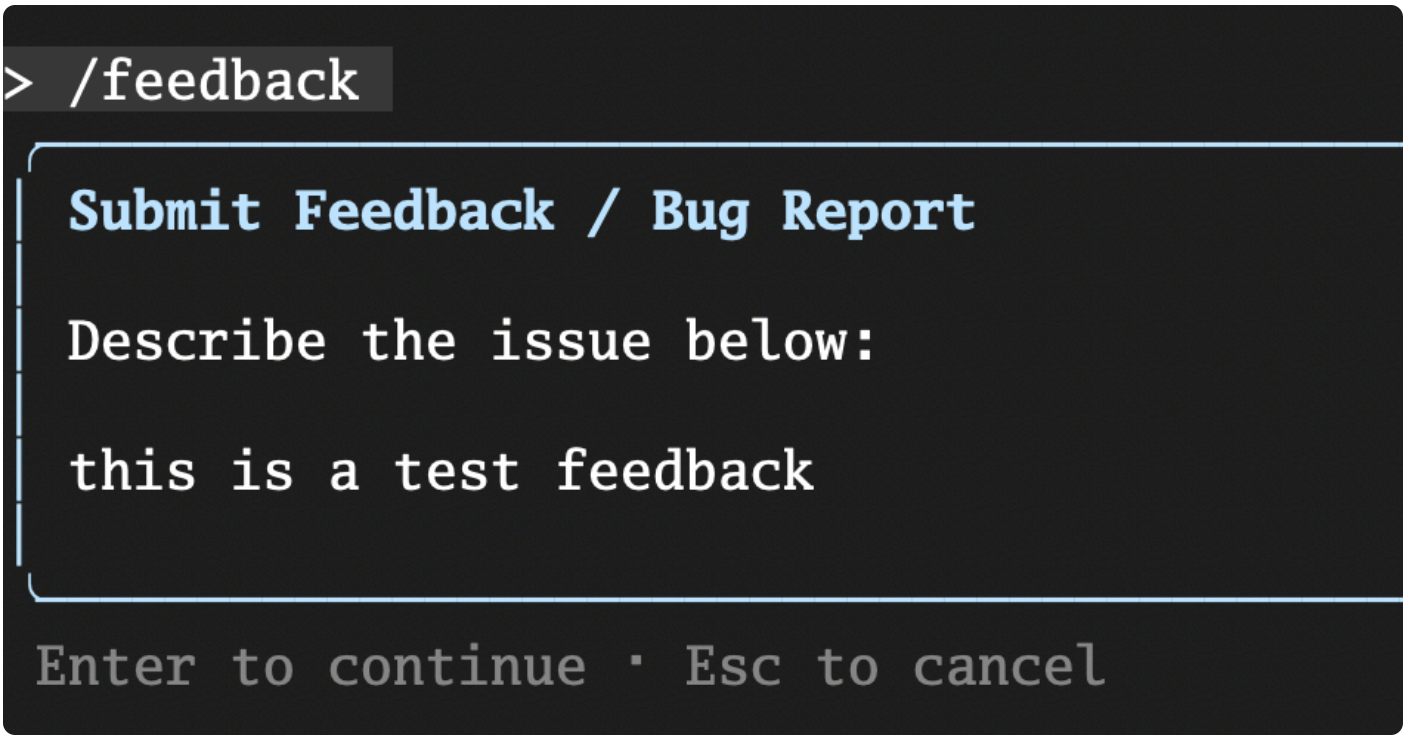
Model: Default (glm-4.6)
Memory: project (CLAUDE.md)
Setting sources: User settings, Shared project settings, Local, Command line arguments, Enterprise managed policies
Esc to exit
```

```
> /doctor

Diagnostics
└ Currently running: npm-global (2.0.15)
└ Path: /root/.nvm/versions/node/v22.16.0/bin/node
└ Invoked: /root/.nvm/versions/node/v22.16.0/bin/claude
└ Config install method: global
└ Auto-updates enabled: default (true)
└ Update permissions: Yes
└ Search: OK (vendor)

Press Enter to continue…
```

```
> /cost
  └ Total cost:            $0.1052 (costs may be inaccurate due to usage of unknown models)
    Total duration (API):  25.6s
    Total duration (wall): 11m 5s
    Total code changes:    0 lines added, 0 lines removed
    Usage by model:
            glm-4.5-air:  20.0k input, 664 output, 117.9k cache read, 0 cache write ($0.1052)
```



熟练地将这些内置指令融入你的肌肉记忆，你的 AI 协作效率就已经能超越大多数人了。但真正的革命，发生在我们从“使用指令”转向“创造指令”的那一刻。

自定义指令：从“使用者”到“指令设计师”

自定义 Slash Commands，是 AI 原生开发范式中，开发者角色从“任务执行者”转向“工作流指挥家”的**第一个、也是最重要的一步**。它让你能够将自己的经验、团队的最佳实践，沉淀为 AI 可以理解和执行的标准化指令。

核心理念：将“高频工作流”封装为斜杠指令

让我们回到开篇时提到的那几个场景。它们有三个共同点：一是高频，你每天可能都要做好几次，甚至更多。二是模式化，每次执行的步骤和核心意图都高度相似。三是蕴含隐性知识，比如“符合我们团队规范”这个要求，背后包含了大量的上下文和约束。

自定义指令的核心，就是将这些模式化的、蕴含隐性知识的高频工作流，从一个需要你每次都详细描述的“**动态 Prompt**”，变成一个固化下来、可一键调用的“**静态模板**”。

Project 级与 User 级指令：团队共享与个人专属

Claude Code 为自定义指令提供了两级作用域，完美地平衡了团队协作和个人效率。

Project 级指令（团队共享）

存放位置：项目根目录下的 `./.claude/commands/` 目录。

特点：这些指令文件（`.md` 格式）会随着你的项目一起被提交到 Git 仓库。团队中的任何成员，只要克隆了项目，就能立即使用这些指令。在 `/help` 菜单中，它们会以 `(project)` 或 `(project:子目录)` 的形式被标记。

场景：极度适合封装团队的公共工作流和最佳实践。例如：`/review-code`, `/gen-test-for-service`, `/deploy-to-staging` 等。

User 级指令（个人专属）

存放位置：你用户主目录下的 `~/.claude/commands/` 目录。

特点：这些指令只存在于你自己的机器上，与任何具体项目无关，可以在你所有的项目中随时调用。在 `/help` 菜单中，它们会以 `(gitignored)` 的形式被标记。

场景：非常适合封装你个人的高频习惯和常用工具集。例如：`/summarize-diff`, `/find-todos-in-code`, `/translate-to-english` 等。



参数的力量：让你的指令“活”起来

如果自定义指令只是一个静态的文本模板，那它的威力将大打折扣。真正让它变得强大的，是其灵活的参数系统。自定义指令的参数有两种形式，我们逐一看一下。

`$ARGUMENTS`：捕获所有

这个占位符会被替换为你在指令名之后输入的所有文本。比如：我们下面创建一个用于修复 GitHub Issue 的指令 fix-github-issue：`./.claude/commands/fix-github-issue.md`

```
Please analyze and fix the GitHub issue: $ARGUMENTS.

Follow these steps:
1. Use `gh issue view` to get the issue details.
2. Understand the problem described in the issue.
3. Search the codebase for relevant files.
4. Implement the necessary changes to fix the issue.
5. Write and run tests to verify the fix.
6. Ensure code passes linting and type checking.
7. Create a descriptive commit message.
8. Push and create a PR.

Remember to use the GitHub CLI (`gh`) for all GitHub-related tasks.
```

当我们像下面这样调用 fix-github-issue 指令时：

```
> /fix-github-issue 1234
```

AI 收到的最终 Prompt：`Please analyze and fix the GitHub issue: 1234...`

`$1、2、3…`：捕获单个

这类参数类似于 Shell 脚本的位置参数，`$1` 代表第一个参数，2 代表第二个，以此类推。参数之间用空格分隔。我们创建一个更结构化的 PR 审查指令来看看这类参数的用法：

`./.claude/commands/review-pr.md`

```
Please review Pull Request #$1.
```

```
2 The priority for this review is: $2.
3 Please focus your review on the changes made by author: $3.
```

当我们像下面这样调用 review-pr 时：

复制代码

```
1 > /review-pr 456 high tonybai
```

AI 收到的最终 Prompt 为：`Please review Pull Request #456. The priority for this review is: high. Please focus your review on the changes made by author: tonybai.`

$ARGUMENTS 和位置参数这两种形式在日常使用时，也是很好选择的。

> 当你的指令需要接收的是一段不确定长度的、完整的自然语言描述或单一 ID 时，使用 $ARGUMENTS。
>
> 当你的指令需要接收的是几个固定的、有明确顺序和含义的结构化参数时，使用位置参数 $1、2、3...。

> 位置参数 $2 就是第二位
> 位置参数 $3 就是第三位

接下来，我们再来看两个自定义斜杠指令的高级技巧。

高级技巧（一）：利用 Frontmatter 定义元数据

对于更专业的指令，你可以在 Markdown 文件的顶部，使用 YAML Frontmatter（即由 `---` 包围的区域）来定义更丰富的元数据，比如下面这个 `/gen-test-for-service` 的例子：

复制代码

```
1 ---
2 description: 为指定的Go函数生成符合团队规范的单元测试。
```

```
argument-hint: [file_path] [function_name]
model: opus
allowed-tools: Bash(go test:*), Write
---

根据我们在 `constitution.md` 中定义的“测试先行”和“表格驱动”原则，为 `@<!--§§MATH_0§§-->2

测试文件应该命名为 `XXX_test.go`。
...
```

让我们来解读一下 Frontmatter 区域使用的这些高级选项：

`description`：定义了当你在 `/` 菜单中看到这个指令时，它旁边显示的描述文字。



`argument-hint`：定义了参数的提示信息。当用户输入 `/gen-test-for-service` 并按下空格时，Claude Code 会提示他接下来应该输入 `[file_path]` 和 `[function_name]`。



`model`：允许你为这个特定的指令**强制指定一个 AI 模型**。

`allowed-tools`：为这个指令**单独设置一套权限**。这是一个极其强大的安全和规范化工具。

通过 Frontmatter，我们为指令赋予了丰富的元数据和清晰的“签名”。但这还不够，一个真正强大的指令，不仅应该能“说”，还应该能“做”。如果我们的指令在执行前，需要先从本地环境中动态地获取一些实时信息呢？接下来，我们就来看看自定义斜杠指令的第二个高级技巧。

高级技巧（二）：在指令中调用 Shell 命令!

这是自定义指令最强大的功能之一：你可以在指令模板内部，**嵌入 `!` 前缀的 Shell 命令**。当指令被调用时，这些 Shell 命令会**首先被执行**，其**输出结果会替换掉命令本身**，成为最终喂给 AI 的上下文的一部分。

下面我们就以**创建一个智能的 Commit Message 生成指令**为例，看看如何在指令中调用 Shell 命令。让我们先创建一个 `/commit` 指令（`./.claude/commands/commit.md`），它能自动抓取当前的代码变更，并让 AI 生成提交信息。

复制代码

```
---
description: 根据当前暂存区的代码变更，生成一条符合Conventional Commits规范的Commit Mess
allowed-tools: Bash(git add:*)
---

你是一位Git专家。请根据以下代码变更的diff信息，为我生成一条符合Conventional Commits规范的、

**当前分支:**
!`git branch --show-current`

**暂存区变更 (Staged Changes):**
!`git diff --staged`

请只输出commit message本身，不要有任何额外的解释。
```

该斜杠命令的调用与执行流程大致如下：

1. 你在 Claude Code 终端中输入：`> /commit`。
2. Claude Code 首先解析指令，发现了两个 `!` 命令。
3. 它执行 `git branch --show-current`，假设输出是 `feature/new-api`。

4. 它执行 `git diff --staged`，假设输出是几百行的代码 diff。

5. 它将这两个命令的输出，替换回模板中的占位符。

6. 最终，它发送给 AI 的完整 Prompt 是：

```
你是一位Git专家。请根据以下代码变更的diff信息...

**当前分支:**
feature/new-api

**暂存区变更 (Staged Changes):**
<此处是几百行的代码diff>

请只输出commit message本身...
```

7. AI 基于这些实时、精准的上下文，为你生成一条完美的 Commit Message。

看到了吗？通过在指令中嵌入 `!`，我们实现了一次**上下文的自动化准备**。这比手动运行命令再复制粘贴，效率提升了不止一个数量级。

实战：创建一个 `/review-go-code` 的团队指令

理论已经足够，现在让我们将所有知识融会贯通，为我的 Go 项目 `bigwhite/issue2md` 创建一个强大、实用、可被整个团队共享的 `/review-go-code` 指令。这个实战将不仅仅是创建一个快捷方式，更是将我们前面学到的上下文艺术、指令设计哲学的一次综合演练。

第一步：明确我们的“工作流”

在动手之前，我们先来设计一下我们理想中的“AI 代码审查”工作流应该是什么样的。一个好的审查，不应该仅仅是让 AI“凭感觉”去看代码，而应该是一个结合了**确定性工具**和 **AI 深度分析**的系统化过程。

我们的工作流设计如下：

1. 开发者提供需要审查的代码路径。
2. 系统首先运行 Go 语言官方的静态检查工具 `go vet`，对代码进行一次快速的、确定性的“体检”，捕获明显的语法和逻辑问题。
3. 然后，将 `go vet` 的检查报告，连同代码本身，以及我们项目最重要的 `constitution.md`（**开发宪法**），一并作为上下文提供给 AI。
4. 最后，AI 在充分理解了所有这些信息的基础上，扮演“首席架构师”的角色，从更高维度的设计原则和最佳实践层面，给出一份深度审查报告。

第二步：创建指令文件

根据我们设计的流程，我们在项目根目录下，创建这个团队共享的指令文件：`./.claude/commands/review-go-code.md`。

第三步：编写指令的“源代码”

将以下内容完整地粘贴到我们刚刚创建的 `./.claude/commands/review-go-code.md` 文件中：

复制代码

```
---
description: 审查指定的Go代码文件或目录，并根据项目“宪法”和静态检查工具给出反馈。
argument-hint: [path_to_review]
model: opus
allowed-tools: Read, Grep, Glob, Bash(go vet:*)
---

你现在是 `issue2md` 项目的首席架构师，你的任务是审查一段代码。

**审查目标：**
请仔细阅读并分析`@$1`中的所有Go代码。

**静态检查初步分析结果：**
这是`go vet`工具对目标路径的检查报告。如果报告为空，则代表没有发现明显问题：
!`go vet $1`

**审查准则（必须严格遵守）：**
请结合上面的静态检查结果，并根据我们项目根目录下的`constitution.md`文件定义的开发宪法，对代码进

1.  **第一条：简单性原则。** 代码是否存在不必要的抽象？是否优先使用了标准库？
```

```
2.  **第三条：明确性原则。** 错误处理是否都使用了`fmt.Errorf("...: %w", err)`进行包装？是否
3.  **第四条：单一职责原则。** 包、文件和函数的职责是否足够单一和内聚？

**输出格式：**
请以Markdown格式，生成一份结构化的审查报告。报告应包含以下部分：
- **总体评价：** 一句话总结代码的整体质量。
- **优点（做得好的地方）：** 列出1-2个最值得称赞的点。
- **待改进项（按优先级排序）：**
  - **[高优先级]:** 必须修改的问题（例如，违反了“宪法”的不可协商条款或`go vet`报告的错误）。
  - **[中优先级]:** 强烈建议修改的问题（例如，设计模式可以优化）。
  - **[低优先级]:** 一些代码风格或可读性的建议。

对于每一个待改进项，请明确指出**文件名、行号、问题描述**，并给出具体的**修改建议**。
```

第四步：深度剖析我们的指令

让我们来仔细剖析这份指令的每一部分，理解其背后的设计哲学：

Frontmatter 部分：

`description`：“审查指定的 Go 代码文件或目录...” —— 给了用户清晰的指引。

`argument-hint`：`[path_to_review]` —— 告诉用户这个指令需要一个路径参数。

`model: opus` —— 我们强制将这次审查任务交给我们最强大的 `opus` 模型，确保审查的深度和质量，这正是“好钢用在刀刃上”。

`allowed-tools: Read, Grep, Glob, Bash(go vet:*)` —— 这是设计的精髓所在。我们为这个指令设定了**最小权限原则**。它只能进行**只读操作**（`Read`, `Grep`, `Glob`），以及**唯一一个被允许的、安全的 Shell 命令**——`go vet`。AI 在这个指令下，绝对无法执行 `rm`、`git` 或任何其他可能产生副作用的命令。

指令主体部分：

指令主体		目的
角色设定	你现在是issue2md项目的首席架构师...	再次强调，为AI设定一个清晰、专业的角色，能极大地提升其输出质量
上下文组合	指令的核心是@$1 和 !go vet $1 的组合	@$1 让AI读取了代码的内容，而 !go vet $1 则动态地为其注入了静态分析的结果。这是一个典型的“人机协作”升级为“机机协作再由人指导”的高级模式
锚定“宪法	请...根据我们项目根目录下的constitution.md文件...	这句话是整个指令的“灵魂”。它将AI的审查标准，从它自己通用的、泛泛的“知识”，精准地锚定到了我们项目独有的、不可动摇的核心原则上。AI的反馈将不再是“正确的废话”，而是真正贴合我们项目哲学的深度洞见
结构化输出	请以Markdown格式，生成一份结构化的审查报告...	我们没有让AI自由发挥，而是为它定义了清晰的输出 Schema（总体评价、优点、待改进项）。这保证了每次审查报告的格式都是一致的、可预测的，便于团队阅读和后续的自动化处理

极客时间

第五步：保存并测试

现在，这个强大的团队能力已经被固化下来了。打开一个新的 Claude Code 会话（确保它能加载到我们新创建的指令），然后尝试调用它：

```
> /review-go-code is running… internal/converter/
  └ Model: glm-4.6
  └ Allowed 4 tools for this command

● I'll review the Go code in the internal/converter/ directory as the chief architect of the issue2md project. Let me start by
  examining the code and checking for any static analysis issues.

● Bash(go vet ./internal/converter/...)
  └ (No content)

● Read(constitution.md)
  └ Read 49 lines

● Search(pattern: "internal/converter/*.go")
  └ Found 1 file (ctrl+o to expand)

● Read(internal/converter/converter.go)
  └ Read 116 lines

● issue2md 项目代码审查报告

  总体评价

  internal/converter/ 包的代码质量整体较高，体现了良好的Go语言实践，但在错误处理和代码重复方面存在一些需要改进的地方。
```

从图中，我们看到，Claude Code 自动地、一气呵成地完成我们设计的整个工作流：

1. 切换到默认模型。
2. 执行 `go vet internal/converter/` 命令，并将结果呈现在上下文中。
3. 读取 `internal/converter/` 目录下的所有 Go 文件，以及项目根目录下的 `constitution.md` 文件。
4. 最终，在充分消化了所有这些信息之后，为你输出一份完全按照我们定义的格式、围绕我们定义的“宪法”原则展开的、高质量的代码审查报告。

复制代码

```
● issue2md 项目代码审查报告

  总体评价

  internal/converter/ 包的代码质量整体较高，体现了良好的Go语言实践，但在错误处理和代码重复方

  优点（做得好的地方）

  1.简单性原则体现良好：代码结构清晰，没有过度抽象，优先使用了标准库（strings.Builder、fmt、s
  2. 单一职责原则执行到位：converter包专注于将GitHub数据结构转换为Markdown格式，职责明确且内

  待改进项（按优先级排序）

  [高优先级]: 缺少错误处理机制

  文件: internal/converter/converter.go
  行号: 全部函数
  问题描述: 所有转换函数（IssueToMarkdown、PullRequestToMarkdown、DiscussionToMarkdow
  数据验证等。这违反了宪法第三条关于错误处理的明确规定。

  修改建议:
  func IssueToMarkdown(issue *github.Issue, comments []github.Comment, enableUser
      if issue == nil {
          return "", fmt.Errorf("issue cannot be nil")
      }
      if issue.User.Login == "" {
          return "", fmt.Errorf("issue user login cannot be empty")
      }

      var sb strings.Builder
      // ... 其余代码保持不变
      return sb.String(), nil
```

```
}

[中优先级]: 代码重复严重

文件: internal/converter/converter.go
行号: 55-115
问题描述: IssueToMarkdown、PullRequestToMarkdown和DiscussionToMarkdown三个函数存在

修改建议:
type ItemType interface {
    *github.Issue | *github.PullRequest | *github.Discussion
}

func itemToMarkdown[T ItemType](item T, itemType string, comments []github.Comm
    var sb strings.Builder

    sb.WriteString(fmt.Sprintf("# %s\n\n", item.getTitle()))
    sb.WriteString(fmt.Sprintf("- **%s Number**: #%d\n", itemType, item.getNumb
    sb.WriteString(fmt.Sprintf("- **URL**: %s\n", item.getURL()))
    sb.WriteString(fmt.Sprintf("- **Created by**: %s\n\n", formatUser(item.getU
    sb.WriteString(fmt.Sprintf("## Description\n\n%s\n\n", item.getBody()))
    writeReactions(&sb, item.getReactions(), enableUserLinks)

    if len(comments) > 0 {
        sb.WriteString("## Comments\n\n")
        for i, comment := range comments {
            writeComment(&sb, i, comment.User, comment.Body, enableUserLinks)
            writeReactions(&sb, comment.Reactions, enableUserLinks)
        }
    }

    return sb.String(), nil
}

[中优先级]: 缺少公共API文档

文件: internal/converter/converter.go
行号: 55, 76, 97
问题描述: 所有公共函数都缺少GoDoc注释，违反了宪法第三条关于"所有公共API都必须有清晰的GoDoc注

修改建议:
// IssueToMarkdown converts a GitHub issue and its comments to Markdown format.
// It returns the formatted Markdown string or an error if the input is invalid
func IssueToMarkdown(issue *github.Issue, comments []github.Comment, enableUser
    // ... 实现
}

[低优先级]: 常量定义可以更灵活

```

```
文件: internal/converter/converter.go
行号: 11
问题描述: githubBaseURL硬编码为"https://github.com"，缺少灵活性，虽然当前符合需求，但考虑
Enterprise的可能性，建议可配置化。

修改建议:
var githubBaseURL = strings.TrimSpace(getEnvOrDefault("GITHUB_BASE_URL", "https

总结

该包整体遵循了项目的核心开发原则，特别是在简单性和单一职责方面表现优秀。主要的改进空间在于增加
复。建议优先解决错误处理问题，然后重构重复代码以提高可维护性。
```

我们成功了！我们不仅创造了一个快捷方式，更是**将团队最高水平的代码审查经验，固化成了一个任何成员（包括新成员）都可以一键调用的原子能力**。这，就是自定义指令的真正威力。

当然，除了我们今天学习的、基于 Markdown 文件的自定义指令，Claude Code 的指令系统还支持一种更高级的形式，即由**插件（Plugin）或MCP 服务器**动态提供的指令，例如 `/mcp__github__list_prs`。这类指令通常具备更强的动态交互和深度集成能力。不过，它们属于更高阶的扩展机制，我们将在后续讲解 MCP 的章节中再深入探讨。对于绝大多数日常工作流的封装，我们今天所学的斜杠指令已经绰绰有余，并且是理解所有指令系统的坚实基础。

本讲小结

今天，我们深入探索了 Claude Code 的指令中枢——Slash Commands。我们从一个被动的 AI“提问者”，开始向一个主动的“指令设计师”转变。

我们系统性地学习了**内置指令**这一“瑞士军刀”，并将其分为会话与上下文管理、环境与配置、项目与协作、原信息与帮助四大类进行深度剖析。

我们深入了**自定义指令**的核心理念，理解了如何将高频工作流封装为原子能力，并通过 **Project 级**和 **User 级**指令，实现了团队协作与个人效率的平衡。然后，我们掌握了让指令“活”起来的关键——**参数系统**，以及通过 Frontmatter（定义元数据）和嵌入 Shell 命令（`!`）进行高级定制的强大技巧。

最后，通过一个完整的实战，我们亲手为团队打造了一个升级版的 `/review-go-code` 指令，将静态检查与 AI 的深度分析完美结合，将团队的最佳实践沉淀为了可复用的 AI 能力。

Slash Commands 是你将 AI 原生开发理念落地的第一块、也是最重要的一块基石。它让你能够开始“编程”AI，将你的智慧和经验，转化为 AI 可以理解和执行的指令。然而，我们今天创建的指令，其能力边界仍然受限于 Claude Code 的内置工具集和我们授权的 Shell 命令。如果我们想让 AI 执行一些更复杂、更专业的操作，比如连接数据库、调用一个内部 API、或者执行一个复杂的安全扫描脚本，该怎么办呢?

这就是我们进阶篇要解决的核心问题。从下一讲开始，我们将进入 AI 能力扩展的深水区，首先从所有 AI Agent 都必须面对的基础——**安全**开始。我们将学习如何通过沙箱和权限控制，为 AI 戴上“安全镣铐”，为我们后续更大胆的能力扩展，奠定坚实的基础。

思考题

今天我们创建了一个强大的 `/commit` 指令，它能自动读取 `git diff --staged`。但它有一个小小的缺憾：如果开发者忘记了 `git add`，那么 `git diff --staged` 的输出就是空的，AI 生成的 Commit Message 也会不准确。请你思考一下，如何**改进**这个 `/commit` 指令，让它变得更“智能”、更“健壮”?

提示：

1. 你可以在指令的 Markdown 文件中，使用**自然语言**来描述**条件逻辑**（比如 “如果...那么...否则...”）。
2. 你可以组合使用**多个 `!` 命令**来获取不同的状态（比如，同时检查 `git diff --staged` 和 `git diff`）。

请尝试设计一个“防呆版”的 `/commit` 指令。当它发现暂存区为空，但工作区有修改时，它应该如何智能地与用户交互？欢迎在评论区分享你的指令设计，看看谁的方案最高效、最优雅！如果你觉得这节课的内容对你有帮助的话，也欢迎你分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (11)

最新　精选



李思锋
2025-12-05 来自中国台湾

---
description: 根据当前暂存区的代码变更，生成一条符合Conventional Commits规范的Commit Message。
allowed-tools: Bash(git add:*)

你是一位资深的Git专家。请根据以下提供的 **Git上下文信息**，按照逻辑判断并生成内容。

Git 上下文信息

1. 当前分支 (Current Branch):
!`git branch --show-current`

2. 文件状态概览 (File Status):
!`git status --short`

3. 暂存区变更 (Staged Changes - 优先):
!`git diff --staged`

4. 工作区未暂存变更 (Unstaged Changes - 备用):
!`git diff`

生成逻辑与要求

请按照以下优先级顺序进行判断和输出：

情况 A：如果「暂存区变更」不为空

1. 忽略「工作区未暂存变更」。
2. 仅根据「暂存区变更」的内容，生成一条符合 Conventional Commits 规范的 Commit Message。
3. **输出格式**：直接输出 Commit Message，不需要任何前缀或解释。

情况 B：如果「暂存区变更」为空，但「工作区未暂存变更」或「文件状态概览」不为空
1. 这说明用户忘记执行 `git add` 了。
2. 根据「工作区未暂存变更」的内容，生成一条 **建议的** Commit Message。
3. **重要**：在输出 Commit Message 之前，必须先用一行加粗文字提示用户：**"暂存区为空，以下是根据工作区变更生成的建议（请先 git add）："**

情况 C：如果所有变更都为空
1. 直接回复："当前工作区是干净的，没有检测到任何代码变更。"

Commit Message 规范
- 格式：`<type>(<scope>): <subject>`
- 常用Type：feat, fix, docs, style, refactor, perf, test, build, ci, chore, revert
- 语言：请使用中文（除非项目主要语言为英文）

请只输出commit message本身，不要有任何额外的解释。

作者回复: 👍

共 3 条评论 >

28



jarvis
2025-12-14 来自上海

command继续进化就是现在的skills了，anthropic这帮人真是太聪明了。

作者回复: command是主动调用的, by human。
skill是大模型调用的，by llm。

共 4 条评论 >

22



Kinco.
2026-02-27 来自上海

"对于内置命令如 /help 和 /compact，请参阅交互模式。
自定义斜杠命令已合并到 skills 中。 .claude/commands/review.md 中的文件和 .claude/skills/review/SKILL.md 中的 skill 都会创建 /review 并以相同方式工作。你现有的 .claude/commands/ 文件继续工作。Skills 添加了可选功能：支持文件的目录、控制你或 Claude 是否调用它们的 frontmatter，以及 Claude 在相关时自动加载它们的能力。"
https://code.claude.com/docs/zh-CN/skills，老师，现在 command 已经合并到 skill 中了。

作者回复: 是的。cc的演进很快。以后都以skill为中心了。

6



极客酱酱
2025-12-20 来自四川

Claude 的 SlashCommand 工具已被移除。“/commit”和“/review-pr 123”现在被视为技能调用。技能调用可以包含可选参数，并且明确告知 Claude “/...” 语法指的是技能。

作者回复: 我理解是不是“换了个称呼”:)

2



sheep
2025-12-09 来自广东

---
description: 根据当前暂存区的代码变更，生成一条符合Conventional Commits规范的Commit Message。
allowed-tools: Bash(git add:*)

你是一位Git专家。请根据以下代码变更的diff信息，为我生成一条符合Conventional Commits规范的、高质量的`git commit`中文消息。

当前分支:
!`git branch --show-current`

暂存区变更 (Staged Changes):
!`git diff --staged`

请只输出commit message本身，不要有任何额外的解释和操作。

运行结果

> /clear
  ⎿ (no content)

> /commit is running…
  ⎿ Allowed 1 tools for this command
  ⎿ Bash(git add:*)

> /commit is running…
  ⎿ Allowed 1 tools for this command
  ⎿ Bash(git add:*)

⏺ Bash(git diff --staged)
  ⎿ diff --git a/main.go b/main.go
    index 3da7d32..6cd4f14 100644
    --- a/main.go
    +++ b/main.go
    @@ -2,6 +2,7 @@ package main

     import (
    "bytes"
    + "fmt"
    "github.com/fsnotify/fsnotify"
    "github.com/gin-gonic/gin"
    "github.com/prometheus/client_golang/prometheus/promhttp"
    @@ -86,6 +87,7 @@ func initViperWatch() {
       }
       val := viper.Get("test.key")
       log.Println(val)
    + fmt.Println(viper.GetString("test.key"))
     }

     func initViperV1() {

Bash(git commit –m "feat(main.go): 添加fmt.Println用于viper配置值打印")
└ Interrupted · What should Claude do instead?

我已经告诉他不要执行其他操作了，它还是想帮我进行git commit，这是什么问题呢？模型问题还是提示词有问题呢

作者回复: 这主要与AI 的行为有关：claude Code以及背后的大模型 在分析了 diff 后，认为既然是生成 commit message，那么顺手把 git commit 也执行了是“最自然”的下一步（这是它的智能所在）。

但当它试图执行 git commit 时，因为你没有在 allowed–tools 里给它这个权限，所以它被拦截了（Interrupted）。

ai本身就是不确定性的。连续两次相同的指令都可能收获不同的结果。

可以试试再 在 Prompt 里加强语气：“绝对不要执行 git commit 命令，只输出文本内容供我复制”。（AI 有时候会比较“热心”:)，需要多强调）。

2



sheep
2025–12–09 来自广东

---
description: 根据当前代码变更生成符合 Conventional Commits 规范的中文 Commit Message。即使开发者忘记 git add，也能智能处理。
allowed–tools: Bash(git add:*)

你是一位资深 Git 工程师。请根据以下代码变更，为我生成一条符合 **Conventional Commits** 规范的、**中文** 的高质量 `git commit` 消息。

请遵循以下规则：

1. **优先使用暂存区（staged）变更**
   – 使用命令：`git diff ––staged`

2. **如果暂存区为空（即 diff 内容为空）**
   – 自动 fallback 使用工作区（unstaged）变更：`git diff`

– 并在 commit message 的正文（body）中加入以下提示（中文）：
```
注意：由于暂存区为空，本次提交信息基于未暂存（unstaged）的代码变更生成。
```

3. **Commit message 输出要求**
– 只输出最终的 commit message（标题 + 正文）
– 不要任何额外解释、说明、提示或多余文字
– 必须使用中文
– Commit 类型由你自动判断（例如：feat / fix / refactor / docs / test / chore / perf 等）
– Commit message 必须符合 Conventional Commits 格式：
```
<type>(<scope>): <简短中文描述>

<可选的详细中文描述>
```

当前分支:
!`git branch --show-current`

暂存区变更（Staged Changes）:
!`git diff --staged`

工作区变更（Unstaged Changes，用于 fallback）:
!`git diff`

请根据上述逻辑生成最终中文 commit message。

作者回复: 👍

1

北风一叶
2025-12-06 来自北京

那怎么样可以生成一个 slash 只能我本人用，并且是本项目用，还不能提交 git呢

作者回复: 用户级的slash command(~/.claude/commands/xxx)是不是可以满足你的要求呢 ?-- 你本人用，项目也可以用，还不提交git。

1



sunshinesDL
2026-03-31 来自河南

Tony 老师好，这节课的内容相当详细，非常感谢您对课程的精心打磨！
我在学习本节的「高级技巧（二）：在指令中中调用 Shell 命令」这小节内容时，对文稿中的 /commit 指令有两处小困惑，想问下老师：
文稿中的 /commit 指令如下：
```
---
description: 根据当前暂存区的代码变更，生成一条符合Conventional Commits规范的Commit Message。
allowed-tools: Bash(git add:*)
---

你是一位Git专家。请根据以下代码变更的diff信息，为我生成一条符合Conventional Commits规范的、高质量的`git commit`消息。

**当前分支:**
!`git branch --show-current`

**暂存区变更 (Staged Changes):**
!`git diff --staged`

请只输出commit message本身，不要有任何额外的解释。
```
学生的疑惑是：
1. 上述指令中的 `git add :*` 是（git add *）的笔误吗?
此外，这里为啥不用 `git add .`（能匹配到隐藏文件和被删除的文件） 呢? 😂😂
2. 我注意到指令中「嵌入 ！ 前缀的 Shell 命令」写为了 「!`git diff --staged`」，我理解这里用 ` ` 包裹 Shell 命令可以增加人类阅读文档时的可读性。可是，如果 AI 直接取这条 Shell 指令在 CC 终端中执行应该会报错吧（原因是无法识别 ``）。因此，应该/可以用 `` 包裹 Shell 指令吗?

两个问题比较基础，希望得到老师的解答，感谢！

作者回复: 目前slash command已经与skill合并了。官方的reference似乎也没有了。但是在老版本的文档中就是这么写的。

在当前的skill文档里还能看到一些：https://code.claude.com/docs/en/skills

---
name: pr-summary
description: Summarize changes in a pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)

Pull request context
- PR diff: !`gh pr diff`
- PR comments: !`gh pr view --comments`
- Changed files: !`gh pr diff --name-only`

Your task
Summarize this pull request...

共 3 条评论>

Geek_77140a
2026-03-15 来自北京

请仔细阅读并分析`@$1`中的所有Go代码 请问 这个占位符什么意思啊

作者回复: 就像 Shell 脚本里的 $1, $2 一样。

当你执行 /review-code internal/ 时，internal/ 就是第一个参数，它会自动填充到指令模板中 $1 的位置。

子衿

2025-12-05 来自浙江

来啦

作者回复: 👏